

AI 해커톤 경진대회



Team: Woo

GitHub: <https://github.com/4thIS/Hackathon-Woo-mjc-club>

명지전문대학



목차

1. 문제 정의 및 기획	3
1.1 우리가 발견한 문제	3
1.2 이렇게 해결했다	3
1.3 타깃과 권한 설계	4
1.4 만들지 않기로 한 것 19건	4
1.5 실제 사용 흐름	5
2. 우리가 직접 그린 UI	5
3. AI 활용 — 교내 인프라 위에서	8
3.1 왜 교내 AI Gateway 인가	8
3.2 활동 글 초안 — 환각을 막는 설계가 핵심	9
3.3 AI pick — 성향 설문으로 동아리 TOP 3	10
3.4 실패 3건과 개선 지표	10
4. AI 협업 개발 — 바이브 코딩	10
단계마다 다른 모델을 골랐다	11
계약을 먼저 못 박고 병렬로 달렸다	11
AI가 틀린 지점과 사람이 개입한 곳	11
5. 기술 구현	12
5.1 아키텍처와 기술 선택	12
5.2 데이터 모델과 API	12
5.3 화면	13
5.4 기술적 난제	14
5.5 품질과 성능	14
6. 실용성 및 활용 가능성	15
6.1 지금 바로 쓸 수 있는 상태다	15
6.2 기대 효과	16
6.3 확장	16
7. 결론	16

1. 문제 정의 및 기획

1.1 우리가 발견한 문제

모집이 에브리타임에 흩어져 있다. 게시판은 새 글이 위로 올라오는 구조라, 3월 첫 주에 올라온 모집 글은 둘째 주면 몇 페이지 뒤로 밀린다. 학기 초를 놓치면 그 동아리에 닿을 방법이 사실상 없어진다. 검색도 동아리 이름을 정확히 알아야 걸리고, 어떤 동아리는 글을 올리고 어떤 동아리는 안 올리니 지금 몇 곳이 부원을 받는지 아무도 모른다. 글쓴이가 정말 그 동아리 사람인지 확인할 방법도 없다.

활동 내용을 한자리에서 볼 수가 없다. 크롤링을 하면서 학교 공식 페이지의 원본 표를 직접 확인했는데, 제공되는 항목이 분야·동아리명·창립년도·목적·주요사업 다섯 개뿐이었다. 활동 내역도, 지금 모집을 하는지도, 사진도 없다. 동아리가 자체적으로 남기는 기록은 단독방이나 개인 SNS에 흩어져 있어 외부인은 볼 수 없다. 결국 신입생 손에 들어오는 정보는 "열정적인 분위기" 같은 소개 문장이 전부다.

두 문제 모두 글쓰기 비용에서 나온다. 사진은 저절로 쌓이지만 그걸 문장으로 옮겨 게시하는 일은 매번 누군가의 30분을 잡아먹는다. 그 30분을 아무도 내지 않으니 기록이 끊기고, 기록이 없으니 보여줄 것도 없어 다시 에브리타임으로 간다.

1.2 이렇게 해결했다

문제	해결	구현
모집 글이 밀려 내려간다	모집을 글이 아니라 동아리가 가진 상태로 만들었다. 상태는 밀려나지 않는다	<code>clubs.recruit_status</code> · 모집 필터
몇 곳이 모집 중인지 모른다	메인 지표에 모집중 동아리 수를 항상 띄운다	<code>GET /api/stats</code>
검색이 안 된다	분야·모집 필터에 이름 검색, 활동 글 전문 검색까지 붙였다	아카이브 · 전체 피드(제목·본문·동아리명·태그)
뭘 할지 모르는 신입생	성향 설문에 답하면 동아리 세 곳을 추려 준다	<code>POST /api/ai/recommend</code>
활동을 한 번에 볼 수 없다	전체 피드(최신순), 동아리 타임라인(시간순), 메인 캐러셀(동아리당 1건)	글 하나가 세 화면에 동시에 걸린다
검증되지 않은 모집	학교 이메일과 인증 메일로 구성원을 확인하고, 동아리 개설은 관리자가 승인한다	<code>@mjc.ac.kr</code> · <code>@on.mjc.ac.kr</code> 만 가입
기록이 끊긴다	사진과 한 줄 메모를 넣으면 AI가 초안을 써 준다	<code>POST /api/ai/draft</code>

앞의 여섯 가지는 기록이 쌓여야 쓸모가 생기고, 기록은 쓰기 쉬워야 쌓인다. 그래서 마지막 줄이 나머지를 떠받친다.

1.3 타깃과 권한 설계

동아리 운영을 그대로 담으려고 권한을 네 축으로 나눴다. 학적 상태(재학·휴학·졸업)는 사람에게 붙고, 멤버십(활동중·OB)과 직책(동아리장·부원), 기수는 사람과 동아리의 조합마다 따로 붙는다. 그래서 한 사람이 A 동아리에서는 12기 부원이면서 B 동아리에서는 3기 동아리장일 수 있다. 졸업생인 OB는 열람과 좋아요만 할 수 있게 뒀다. 추억을 보러 오는 자리이지 운영에 관여하는 자리는 아니라고 봤다.

1.4 만들지 않기로 한 것 19건

기획에서 우리가 가장 오래 이야기한 쪽은 무엇을 넣을지가 아니라 무엇을 빼는가였다.

뺀 것	이유
댓글	신고와 삭제, 분쟁 처리가 통째로 따라온다. 이 서비스는 토론이 아니라 기록이 목적이다
동아리 정원	가입 승인과 모집 상태만으로 충분히 조절된다
사이트 내 알림함	이메일과 '내 신청 현황'이 같은 일을 한다
공용 AI 키	비용과 남용 책임이 한 곳에 몰린다 (3.1)
AI 초안 다시 생성	마음에 안 들면 그 자리에서 고치는 편이 빠르다
원본 분류 재해석	학교가 '학술'로 등록한 동아리를 우리가 옮기는 순간 출처가 우리 판단이 되어 버린다
자동 주기 크롤링	25개가 자주 바뀌지 않는다. 필요할 때 손으로 돌리면 된다
마이그레이션 도구 · 이미지 CDN · JWT 등	해커톤 기간에 얻는 것보다 잃는 시간이 크다

여기에 임원 직책, 동아리 간 쪽지, 사진 수집, 지도교수 수집 등을 더해 모두 19건을 이유와 함께 기획서에 적어 뒀다.

1.5 실제 사용 흐름

MJC Club Archive — 실제 사용 흐름

회원가입부터 활동 기록이 다시 신입생에게 닿기까지, 보라색 테두리는 AI가 관여하는 단계다.



그림 1. 회원가입에서 시작해 동아리 개설과 관리자 승인, 모집, AI pick, 가입 신청과 승인을 거쳐 활동 글이 피드에 걸리기까지. 보라색 테두리가 AI가 관여하는 단계다. 10번에서 6번으로 돌아가는 점선이 이 서비스의 순환이다

2. 우리가 직접 그린 UI

기획 첫 시간에는 코드를 한 줄도 쓰지 않았다. 종이를 펴 놓고 화면부터 그렸다.

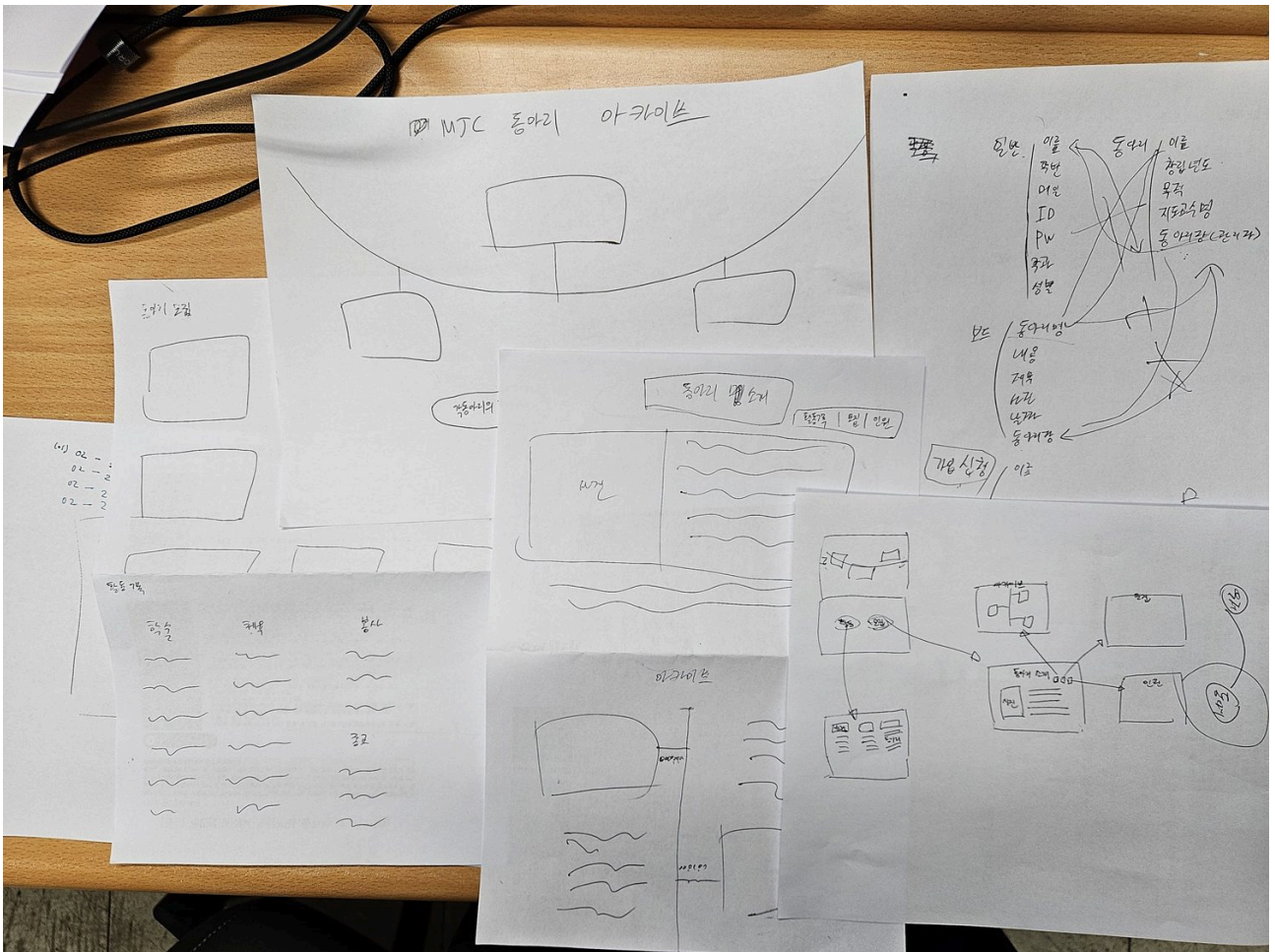
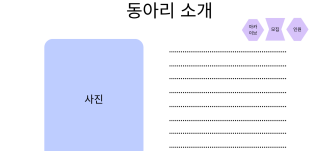


그림 2. 2026-08-06 첫 회의. 화면 구조와 데이터 모델, 화면 흐름을 한 자리에서 잡았다

손그림에 있던 것	최종 구현
제목 아래 곡선에 카드가 매달린 메인	곡선 캐러셀. 기하값을 확정해 그대로 옮겼다
일반 / 동아리 / 보드 / 가입 신청	users · clubs · posts · join_requests. 이름만 바뀌고 필드는 대부분 살아남았다
아카이브 → 동아리 소개 → 인원	흐름이 그대로다
학술 · 체육 · 봉사 세 갈래	실제 학교 데이터를 받고 6종으로 늘렸다

이 그림에서 눈여겨볼 것은 곡선이다. 처음부터 있었고, 끝까지 살아남았다. 우리에게 곡선은 장식이 아니라 시간 축이다. 위에 얹힌 카드가 활동 기록이고 호를 따라 붙은 라벨이 활동 날짜다.

디지털 초안	최종에서 어떻게 됐나																								
 MJC Club Archive mjc hackathon project with Woo MYONGJI COLLEGE 영지전문대학 아카이브 바로가기 ->	곡선과 제목, 학교 로고, '아카이브 바로가기'까지 구조가 그대로 메인이 됐다																								
그림 3. 메인																									
 Archive <table><thead><tr><th>학술</th><th>체육</th><th>계속</th></tr></thead><tbody><tr><td>동아리1</td><td>동아리1</td><td>동아리1</td></tr><tr><td>동아리2</td><td>동아리2</td><td>동아리2</td></tr><tr><td>동아리3</td><td>동아리3</td><td>동아리3</td></tr><tr><td>동아리4</td><td>동아리4</td><td>동아리4</td></tr><tr><td>동아리5</td><td>동아리5</td><td>종교</td></tr><tr><td>동아리6</td><td>동아리6</td><td>동아리6</td></tr><tr><td>동아리7</td><td>동아리7</td><td>동아리7</td></tr></tbody></table>	학술	체육	계속	동아리1	동아리1	동아리1	동아리2	동아리2	동아리2	동아리3	동아리3	동아리3	동아리4	동아리4	동아리4	동아리5	동아리5	종교	동아리6	동아리6	동아리6	동아리7	동아리7	동아리7	세로 나열에서 3열 격자와 필터 레일로 바꿨다. 8개일 때는 나열로 충분했지만 실제 동아리는 25개였다. 분야에 '종교'가 초안부터 들어 있었던 점도 실데이터와 맞아떨어졌다
학술	체육	계속																							
동아리1	동아리1	동아리1																							
동아리2	동아리2	동아리2																							
동아리3	동아리3	동아리3																							
동아리4	동아리4	동아리4																							
동아리5	동아리5	종교																							
동아리6	동아리6	동아리6																							
동아리7	동아리7	동아리7																							
그림 4. 아카이브																									
 세션 3 2024.03.01 2024.03.07 2024.07.28 세션 1	세로 축에 카드를 좌우로 번갈아 붙이고 날짜를 다는 구조. 타임라인이 이 그림 그대로다																								
그림 5. 활동 기록																									
 동아리 소개 사진	아카이브/모집/인원 탭을 없애고 한 화면에 펼쳤다. 기록을 보여주려고 만든 서비스에서 기록을 한 번 더 누르게 할 이유가 없었다																								
그림 6. 동아리 소개																									

옮기는 순서도 정해 뒀다. 목업을 순수 HTML/CSS/JS로 4종(1,660줄) 먼저 만들고, 거기서 나온 수치를 확정값으로 못 박은 다음 Vue로 이식했다. 색과 간격은 `theme.css` 토큰만 쓰기로 하고 컴포넌트에 hex를 적는 것을 금지했다. 캐러셀 기하는 R 1500, tilt 0.42, step 268px로 고정했고 아카이프는 36px 베이스라인 격자에 맞췄다. 모달은 전부 네이티브 `<dialog>`를 썼는데 ESC와 포커스 트랩, 스크롤 잠금을 브라우저가 알아서 해 주기 때문이다. 등장 연출은 아카이프와 피드, 캐러셀이 같은 곡선(`blur 7px→0`, 0.62s)을 쓰도록 맞췄다.



그림 7. 서비스 로고. 이미지 생성 AI Nano Banana로 만들었다

팀에 디자이너가 없어서 로고는 이미지 생성 AI(Nano Banana)로 만들었다. 조건은 세 가지였다. MCA 글자가 읽힐 것, 가운데에 학교 상징인 나무가 들어갈 것, 초승달이 c 자리를 대신할 것. 색은 테마의 `--accent (#7C6BC4)` 계열로 맞춰 로고만 따로 노는 일이 없게 했다. 이 원본에서 헤더 마크와 큰 로고, 파비콘 3종을 뽑아 썼다.

텍스트 생성은 교내 Gateway로, 이미지 생성은 외부 도구로 나뉘었다. 교내 Gateway가 이미지 생성을 제공하지 않고, 로고는 서비스가 돌면서 쓰는 기능이 아니라 개발 중 한 번 만들고 끝나는 산출물이기 때문이다. 서비스가 실행 중에 호출하는 AI는 교내 Gateway뿐이다.

3. AI 활용 — 교내 인프라 위에서

3.1 왜 교내 AI Gateway 인가

우리는 명지전문대학 AI Gateway를 쓰기로 했다. 외부 API는 결제하지 않았다.

후보	장점	단점	채택
명지전문대 AI Gateway	학교 인프라라 별도 결제가 없다. 모델 80종, 비전 지원	크레딧 한도 안에서 써야 한다 (재학생 월 10,000, 매월 초기화)	✓
OpenAI · Anthropic 직접 결제	문서와 SDK가 성숙하다	팀이 비용을 부담해야 하고, 운영에 들어가면 비용이 서비스 한 곳에 몰린다	✗
로컬 오픈 모델	비용이 들지 않는다	시연 장비에서 비전과 한국어 품질을 확보하기 어렵다	✗

남는 크레딧이 이 서비스의 연료다. 학교는 재학생에게 매달 10,000 크레딧을 주고 월 1회 초기화한다. 그런데 이걸 다 쓰는 학생은 드물다. 우리 학교만 놓고 보면 크레딧을 남기는 학생이 과반이다. 결국 예산은 이미 배정돼 있고 쓰이지 않을 뿐인데, 이 서비스가 그 크레딧을 쓸 자리를 만들어 준다. 동아리장은 활동 글을 쓸 때, 신입생은 AI pick을 누를 때 자기 크레딧을 쓴다. 교내 AI 인프라 입장에서는 소모처가 하나 생기는 것이고, 서비스 입장에서는 운영비가 0인 채로 AI 기능을 돌리는 셈이다. 양쪽 모두에게 이득이라 이 구조를 골랐다.

바깥으로 나가지 않는 교내 시스템이다. 가입은 학교 이메일로만 되고, 다루는 데이터는 학교 동아리 데이터이며, AI 호출도 학교 게이트웨이로 간다. 구성원이 학교 데이터를 학교 AI로 다루는 닫힌 구조라, 외부 결제 계정이 없어도 학교가 이 서비스를 그대로 운영할 수 있다.

키는 개인이 쥔다. 사용자가 직접 발급해 '내 정보'에 등록하면 서버가 Fernet으로 암호화해 보관한다. 저장한 다음에는 본인에게도 관리자에게도 값을 보여주지 않고, 지우고 새로 넣는 것만 된다. 공용 키를 하나 두는 방식은 처음부터 배제했다. 비용과 남용 책임이 한 곳에 몰리고, 한 사람이 한도를 넘기면 다른 사람까지 막히기 때문이다. 키가 없어도 서비스는 그대로 돌아가고 AI 버튼만 잠긴다.

게이트웨이는 붙이기 전에 실호출로 확인했다. 스모크 5종을 돌려 모델 80종 조회, 인증 헤더 두 방식, 한국어 생성, 이미지 입력, Anthropic 네이티브 경로가 모두 정상임을 봤다. 비전은 좌우 색이 다른 테스트 이미지를 정확히 구분하는지로 확인했다. 최종 선정 모델은 `claude-sonnet-5` 이고, 텍스트만 쓰는 경우 `gpt-5.6-luna`도 문제없이 동작했다.

3.2 활동 글 초안 — 환각을 막는 설계가 핵심

이 기능에서 가장 신경 쓴 부분은 무엇을 쓰게 할지가 아니라 무엇을 쓰지 못하게 할지였다. **사진만 있고 메모가 없으면 초안을 만들지 않는다.** 대신 무슨 활동이었는지 한 줄만 적어 달라고 돌려보낸다 (422 MEMO_REQUIRED).

사진만 주고 글을 쓰게 하면 AI는 "즐거운 시간을 보냈습니다" 같은 문장을 만들어 낸다. 어느 동아리에 붙여도 말이 되는 문장이라 읽는 사람에게 남는 정보가 없고, 사실과 다를 수도 있다. 그래서 역할을 갈랐다. 사실은 메모와 PDF가 대고, 사진은 그 사실을 구체화하는 데만 쓴다. "12명 참여"를 "12명이 필름 카메라를 들고" 정도로 살을 붙이는 선까지다. 이 규칙은 프롬프트에서 빠지지 않도록 테스트로 묶어 뒀다.

- ① 메모만 → "10월 12일 성북동에서 출사를 진행했습니다. 12명이 참여했고..."
- ② 메모 + 사진 2장 → "...각자 필름 카메라를 들고 성북동 일대를 돌며 촬영했습니다."
- ③ 사진만 → 422 MEMO_REQUIRED

②에서 묘사는 붙었지만 인원과 날짜 같은 사실은 부풀지 않았다. 의도한 그대로다. 결과보고서 PDF는 게이트웨이에 파일 업로드가 없어서 서버가 `pypdf`로 텍스트를 먼저 뽑아 프롬프트에 담는다. 실패는 상황별로 갈라 뒀다. 스캔본이라 글자가 안 뽑히면 422 PDF_UNREADABLE, 키가 없으면 403으로 게이트웨이를 아예 부르지 않고, 한도를 넘기면 429, 게이트웨이가 죽으면 502다. 시연 도중 네트워크가 끊기는 경우까지 생각해 `DEMO_FALLBACK=1`로 준비된 초안을 돌려주는 길도 열어 뒀다. 관련 테스트는 17건이다.

3.3 AI pick — 성향 설문으로 동아리 TOP 3

설문에 답하면 교내 AI가 동아리 25개의 실제 데이터(분야·목적·주요사업·활동 기록)를 읽고 세 곳을 골라 준다. 설문 답도 추천 결과도 서버에든 브라우저에든 남기지 않는다. 테스트는 9건이다.

이 기능은 처음 기획에는 아예 없었다. 안건으로 나온 적조차 없다가 **2차 팀 리뷰에서 처음 제안됐다**. 실데이터 25개를 넣고 아카이브를 다 같이 보던 자리였는데, 화면을 보다가 "우리는 이 목록에서 뭘 고를지 아는 사람이지만 신입생은 아니겠다"는 이야기가 나왔다. 분야 필터는 이미 원하는 게 무엇인지 아는 사람에게만 쓸모가 있다. 체육 동아리를 찾는 사람은 필터를 쓰지만, 뭘 할지 모르는 신입생에게 25개는 그냥 25개다. 그래서 추천 알고리즘을 만들자는 게 아니라 출발점을 하나 줘여 주자는 쪽으로 정리했고, 그 자리에서 안건이 되어 새벽에 구현했다.

3.4 실패 3건과 개선 지표

차 수	문제	원인	조치	결과
v1	게이트웨이가 403을 돌려줬다. 키가 잘못된 줄 알았다	기본 UA로 나가서 Cloudflare가 막았다(error 1010)	브라우저형 UA를 고정하고 규칙으로 문서에 남겼다	스모크 5종 5/5 통과
v2	PNG 사진을 넣으면 400으로 거부됐다	media type을 <code>image/jpeg</code> 로 하드코딩해 뒀다	확장자를 믿지 않고 매직 넘버로 판별하게 고쳤다	4개 포맷 통과, 회귀 4건 추가
v3	같은 메모로 뽑아도 문체가 매번 달랐다	프롬프트에 문체를 지정하지 않았다	문체를 한 줄 박고 회귀 테스트로 묶었다	3회 생성 3/3 일관

지표	개선 전	개선 후
게이트웨이 호출 성공	0% (403 차단)	5종 전부 200
지원 이미지 포맷	JPEG만	png · jpeg · gif · webp
문체 일관성	실행마다 흔들림	3/3 일관
AI 관련 테스트	—	26건 (초안 17 · 추천 9)

4. AI 협업 개발 — 바이브 코딩

3인이 13시간 30분 만에 API 52개와 화면 21개, 테스트 205건을 만들었다. 비결이라고 할 만한 게 있다면 AI에게 일을 시킨 방식이 아니라, AI가 일할 수 있는 판을 먼저 깔아 둔 쪽이다.

우리가 쓴 도구는 Claude Code(설계·구현·리뷰), Superpowers 스킴(brainstorming / writing-plans / subagent-driven-development), MCP Playwright(브라우저 실검증), MCP Context7(라이브러리 문서), MCP Obsidian(문서 작성), Nano Banana(로고), `gh` CLI, Docker Compose다.

단계마다 다른 모델을 골랐다

같은 모델을 처음부터 끝까지 쓰지는 않았다. 기획과 구현이 요구하는 게 서로 반대였기 때문이다.

단계	모델	왜 이 모델인가
기획 · 설계	Fable	빨리 답을 받는 것보다 깊게 파고 되묻는 게 중요한 단계다. 문제 정의와 권한 4축, 환각 방지 규칙, 뺨 기능 19건은 한 번에 나오지 않았다. 시간을 들여 근거를 캐물으며 스펙 문서로 굳혔다
구현	Opus 5 (M)	계약이 확정된 뒤로는 속도가 곧 완성도였다. 세 사람이 각자 브랜치에서 동시에 돌려 19~20시에만 커밋 36건을 냈다

기획에 들인 시간이 구현 속도를 만들었다고 본다. 스펙과 계약이 이미 정해져 있으니 구현할 때 되물을 게 없었고, 그래서 빠른 모델로 각자 달려도 결과가 어긋나지 않았다. 모델 선택도 설계 결정의 하나로 다뤘다는 뜻이다.

계약을 먼저 못 박고 병렬로 달렸다

AI로 코드를 빨리 뽑을수록 합칠 때 깨진다. 그래서 코드보다 계약을 먼저 만들었다. 18:56에 docs/api.md(806줄)를 확정했고, 그 직후인 19시대에 커밋 18건, 20시대에 18건이 몰렸다. 프론트는 백엔드가 없는 동안 명세대로 mock을 만들어 화면을 완성했고 나중에 mock만 지웠다. 계약 파일 3종은 합의 없이 못 바꾸게 하고 바꿀 때는 PR 제목에 [계약]을 달았다(10건). 36개 PR을 머지하는 동안 계약이 어긋나서 다시 만든 코드는 없었다.

절차는 스킬로 강제했다. Task마다 구현 에이전트가 만들고 리뷰 에이전트가 검토한 뒤 수정하고, 마지막에 브랜치 전체를 다시 리뷰하는 식이다. 진행 원장과 리뷰 diff 6건이 .superpowers/sdd/에 남아 있다. 검증은 눈으로도 했다. 비로그인 상태에서 가입 신청을 눌렀을 때 모달이 보던 화면 위에 뜨는지, 캐러셀 등장 애니메이션을 240ms 지점에 세워 놓고 띠와 마디, 카드 순서가 맞는지, 애니메이션이 끝난 뒤 호버가 살아 있는지를 실제 브라우저에서 확인했다.

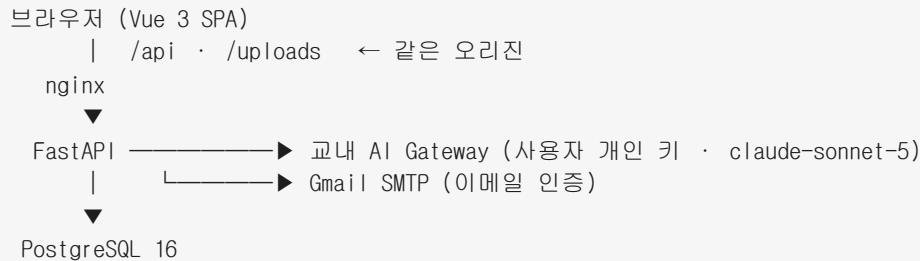
AI가 틀린 지점과 사람이 개입한 곳

지점	AI가 한 것	사람이 잡은 것
크롤러 분류	모르는 분야를 취미 · 교양으로 떨어는 폴백을 넣었다	리뷰 에이전트가 지적했다. 학교가 새 분야를 만들면 조용히 오분류된다. RuntimeError로 바꿨다
'종교' 분야 추가	백엔드만 고쳤다	리뷰에서 프론트 4곳이 빠진 걸 찾았다. 그대로 뒀으면 개선판에서 종교를 고를 수조차 없었다
시드 데이터	데모 학생을 8개 동아리 전부에 가입시켰다	시연 리허설에서 '가입 신청'을 누를 곳이 없다는 걸 발견했다
테스트 픽스처	프로세스 안에서만 세는 카운터로 고유 ID를 만들었다	두 번째 실행에서 터졌다. 원인 분석과 수정은 사람이 했다(5.4)

숫자는 확인할 수 있는 것만 적었다. 커밋 113, PR 36, 테스트 205, 문서 5,371줄. 이 문서의 모든 수치는 커밋과 PR, 직접 측정에서 나왔다.

5. 기술 구현

5.1 아키텍처와 기술 선택



프론트와 백엔드를 같은 오리진에 뒀다. 세션 쿠키가 `httpOnly`에 `SameSite=Lax`라 오리진이 같으면 쿠키 정책과 CORS를 동시에 손봐야 하는데, `/api`를 프록시하면 그 문제가 통째로 사라진다.

계층	선택	이유
백엔드	FastAPI + SQLAlchemy 2 + psycopgp	타입 힌트가 그대로 문서(<code>/api/docs</code>)가 된다
프론트	Vue 3 + Vite + Pinia	목업이 순수 HTML/CSS/JS라 옮기는 비용이 가장 적었다
인증	세션 쿠키 + bcrypt	이 규모에서 JWT는 이득이 없다. 로그아웃이 쿠키 삭제로 끝난다
PDF	pypdf 텍스트 추출	게이트웨이에 파일 업로드가 없어 서버가 먼저 글자를 뽑는다
마이그레이션	<code>create_all</code> + <code>seed.py</code>	해커톤 기간에는 스키마를 빠르게 갈아엎는 편이 이득이다

5.2 데이터 모델과 API

테이블은 10개다. `users`는 학번(숫자 10자리)을 PK로 잡았다. 이메일 도메인이 두 종류라 학번을 키로 두면 중복 가입이 DB에서 막힌다. 학년은 학번에서 유추하지 않고 별도 컬럼으로 뒀는데, 휴학 같은 사정으로 학번과 학년이 어긋나는 경우가 흔해서다. `club_members`에 직책과 멤버십, 기수가 붙고, `leave_requests`는 7일이 지나면 자동 승인되어 승인권자가 사라져도 사용자가 갇히지 않는다. `posts`는 공개 기본값을 `false`로 뒀다. 활동 사진에 학생 얼굴이 들어가기 때문이다.

API는 52개이고, 명세는 `docs/api.md` 806줄에 계약으로 관리한다.

Method	Endpoint	설명	권한
GET	/api/stats · /api/clubs · /api/clubs/{id}	지표 · 목록 · 상세	공개
GET	/api/posts	전체 피드 (최신순 + 검색)	공개
GET	/api/posts/highlights	메인 캐러셀. 동아리당 1건	공개
POST	/api/ai/draft · /api/ai/recommend	활동 글 초안 · AI pick	장 · 인증
POST	/api/clubs/{id}/join-requests → /approve	가입 신청 → 승인(기수 자동 부여)	인증 → 장
POST	/api/admin/club-applications/{id}/approve	개설 승인	관리자

계약으로 못 박은 것이 두 가지 있다. 모든 4xx·5xx는 {"detail":{"code","message"}} 한 형태로만 내려간다. 그리고 권한은 공개·인증·부원·소속·장·관리자 6단계이며 라우터 의존성에서만 검사한다.

5.3 화면

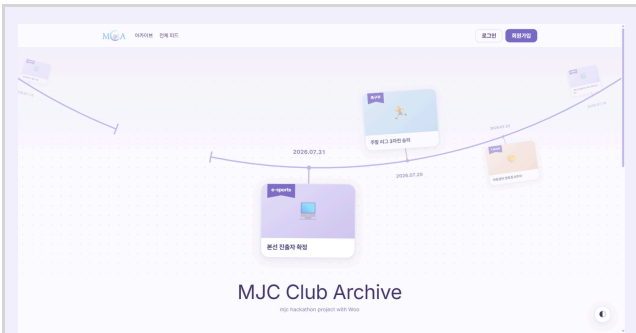


그림 8. 메인. 곡선 캐러셀에 동아리당 1건만 올려 한 곳이 화면을 독점하지 못하게 했다. 드래그와 방향키로 넘기고 손을 떼면 관성으로 미끄러진다

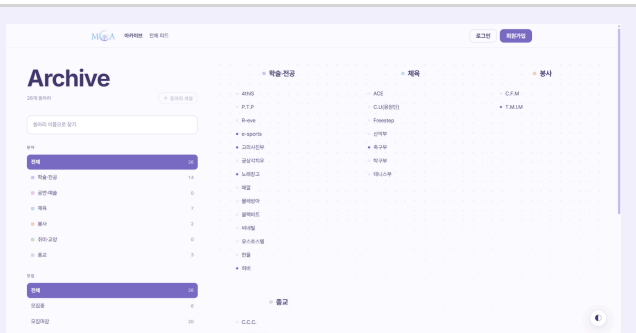


그림 9. 아카이브. 좌측 필터 레일과 분야별 3열 격자. 필터와 검색은 전부 클라이언트에서 계산한다

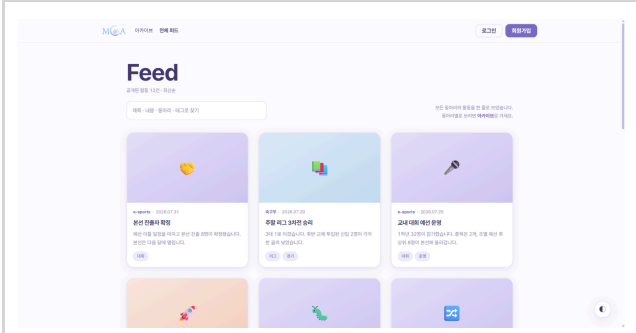


그림 10. 전체 피드. 동아리를 가로질러 최신순으로 흐른다. 남의 동아리 비공개 글은 검색어가 맞아도 나오지 않는다(테스트 3건으로 고정)



그림 11. AI 초안. 메모와 사진, 결과보고서 PDF를 받는다. 생성에 10초쯤 걸려서 진행 상태를 단계별로 보여 준다

5.4 기술적 난제

#	문제	원인	해결
1	테스트를 다시 돌리면 대부분 실패했다. CI는 초록인데 로컬만 터졌다	픽스처 카운터가 프로세스 안에서만 셧다	실행 표식을 붙이고 세션 시작·종료 양쪽에서 정리하게 했다
2	PNG 사진에서 AI 호출이 400	media type 하드코딩	매직 넘버로 판별
3	크롤러가 모르는 분야를 조용히 오분류	폴백이 원칙을 위반했다	<code>RuntimeError</code> 로 즉시 중단
4	'종교'를 개설 폼에서 고를 수 없었다	분야 목록이 프론트 4곳에 하드코딩	단일 출처로 통일
5	시연 계정이 '가입 신청'을 누를 곳이 없었다	시드가 모든 동아리에 가입시켰다	두 곳만 남겼다
6	제출 직전 CI 실패	GitHub Actions 인프라 장애(공식 incident)	CI와 같은 환경을 로컬에 재현해 검증하고 진행했다

가장 애먹은 건 1번이다. 픽스처가 고유 학번을 만들려고 쓰던 카운터가 프로세스 안에서만 세는 물건이라, `pytest`를 새로 띄우면 1부터 다시 세면서 지난 실행이 남긴 학번과 겹쳤다. 두 번째 실행부터 109 passed가 35 failed + 17 errors로 바뀌었다. 여기까지도 성가신 정도인데, 진짜 문제는 그다음이었다. 테스트가 남긴 찌꺼기가 `seed.py`의 "이미 데이터가 있습니다" 판정을 통과시켜 데모 데이터를 넣지 않고 끝냈다. CI는 매번 새 DB를 쓰니 이 문제가 드러날 리 없었다. 그대로 뒀다면 시연 준비 때 seed를 돌리고 완료 메시지만 확인한 뒤 빈 화면으로 시연장에 갔을 것이다. 실행마다 다른 표식을 붙이고, 세션 시작과 종료 양쪽에서 FK 순서대로 정리하도록 고친 뒤 연속 3회 실행 통과와 잔여물 0건을 확인했다.

5.5 품질과 성능

```
$ uv run pytest -q      205 passed, 1 skipped in 83.40s
$ uv run ruff check .    All checks passed!
```

테스트는 22개 파일에 194개 함수, 실행 기준 205건이다. 시연 시나리오에서 화면이 비거나 권한이 새는 경로를 먼저 뚫었고, 계약에 적힌 규칙은 테스트로 묶어 조용히 깨지지 않게 했다. 캐러셀이 동아리당 1건인지, 신청자 정보가 네 필드만 나가는지, AI 프롬프트에 환각 방지 문장이 들어 있는지가 그런 예다. CI는 GitHub Actions에서 `ruff`와 `pytest`(PostgreSQL 서비스 컨테이너), 프론트 빌드를 돌리고 문서만 바뀐 PR은 빌드를 건너뛴다. PR 36건은 전부 리뷰를 거쳐 머지했고 미머지로 남은 건 없다. 세 사람 코드가 처음 합쳐진 시점에는 시연 시나리오 전 구간을 손으로 밟아 20 PASS / 0 FAIL을 확인했다.

엔드포인트 (로컬 도커, 5회 중앙값)	응답	지표	실측
GET /api/clubs?category=...	4.2 ms	프론트 빌드	972 ms
GET /api/clubs/{id}	7.7 ms	초기 번들	gzip 56.7 kB
GET /api/stats	8.4 ms	코드 분할	화면별 13개
GET /api/posts/highlights	10.7 ms	테스트	205 passed / 83초
GET /api/clubs	14.4 ms	예외 처리	단일 에러 형태. 메일 발송이 실패해도 서버는 죽지 않는다

6. 실용성 및 활용 가능성

6.1 지금 바로 쓸 수 있는 상태다

돌아가는 데이터가 데모용이 아니다. 명지전문대학이 실제로 등록한 동아리 25개(학술·전공 13, 체육 7, 종교 3, 봉사 2) 위에서 돈다. 가입과 내 정보의 학과도 학교 홈페이지 목록에서 고른다. 크롤러는 손으로 돌리고 결과를 git에 커밋하는 방식이라, 앱이 실행 중에 학교 서버를 호출하는 일은 없다. 시연 도중 학교 서버가 느려도 영향을 받지 않는다.

도입 장벽	대응
동아리장이 첫 글을 올려야 순환이 시작된다	크롤링으로 25개 동아리의 기본 정보를 미리 채워 빈 목록을 없앴다
AI 키를 개인이 발급해야 한다	키가 없어도 서비스 전체는 정상 동작한다
학교 계정 인증이 필요하다	도메인 2종(mjc.ac.kr · on.mjc.ac.kr)을 모두 허용한다
운영 인력	관리자 화면에서 개설 승인, 동아리장 강제 교체, 유저 관리를 처리한다

6.2 기대 효과

구분	내용
정량	학교 공식 페이지가 주지 않던 모집 상태·활동 기록·가입 경로를 25개 동아리 전체에 제공한다
정량	흩어져 있던 모집 정보가 한 화면의 지표로 모인다. 지금 몇 곳이 모집 중인지 항상 보인다
정성	신입생이 소개글이 아니라 활동 이력을 보고 동아리를 고른다
정성	동아리장이 잠수해도 부원이 갇히지 않는다(탈퇴 7일 자동 승인)
정성	동아리장 공석으로 동아리가 마비되지 않는다(관리자 강제 교체)
정성	남아돌던 교내 AI 크레딧에 쓸 자리가 생긴다

6.3 확장

동아리를 학생회나 학과, 교내 프로그램으로 바꿔도 구조는 그대로 성립한다. 조직이 활동을 기록하고 학생이 그 기록을 보고 참여를 정한다는 뼈대가 같기 때문이다. 권한 4축(학적·멤버십·직책·기수)도 그대로 쓸 수 있다.

시점	과제
단기	Alembic 도입, 업로드 파일 외부 스토리지, 배포
중기	동아리 데이터 갱신 절차 정립, AI 응답 시간 계측, 모바일 최적화
장기	학생회·학과로 대상 확대, 학교 통합 로그인 연동

7. 결론

신입생이 동아리를 찾아 가입하고, 동아리장이 AI로 활동 글을 올리고, 그 글이 다시 신입생에게 닿는 한 바퀴를 13시간 30분 안에 끝까지 돌려 봤다. 데모 데이터가 아니라 명지전문대학이 실제로 등록한 동아리 25개 위에서도.

API 계약을 먼저 확정하고 병렬로 달렸다	19~20시에만 커밋 36건, 36개 PR을 머지하며 계약 문제로 다시 만든 코드 0건
화면을 손으로 먼저 그렸다	첫 회의에서 그린 구조가 13시간 뒤 화면과 그대로 맞았다
뺨 기능 19건을 이유와 함께 남겼다	13시간 30분 내내 범위가 흔들리지 않았다
단계마다 다른 모델을 골랐다	기획은 깊게(Fable), 구현은 빠르게(Opus 5). 기획에 쓴 시간이 구현 속도를 만들었다

실행: `docker compose up --build` → `localhost:8080` · 시드 `docker compose exec backend uv run --no-dev python seed.py` · 데모 계정 `2024260001@mjc.ac.kr` / `test1234`(동아리장) · 저장소 <<https://github.com/4thIS/Hackathon-Woo-mjc-club>>